南開大學
Nankai University

计算机学院

并行程序设计实验报告

CPU 架构相关编程

姓名：崔迪

学号：2412439

专业：计算机科学与技术

课程：并行程序设计

指导教师：崔立骁

完成日期：2026 年 4 月 1 日

目录

一、 基础要求 **2**
（一） 算法设计 2
1. $n*n$ 矩阵与向量内积 2
2. n 个数求和 2
（二） 编程实现 2
1. $n*n$ 矩阵与向量内积 2
2. n 个数求和 3
（三） 性能测试 3
1. $n*n$ 矩阵与向量内积 3
2. n 个数求和 4
（四） profiling 5
1. $n*n$ 矩阵与向量内积 5
2. n 个数求和 5
（五） 结果分析 6
1. $n*n$ 矩阵与向量内积 6
2. n 个数求和 6

二、 进阶要求 **7**
（一） 循环展开优化 7
1. 算法设计 7
2. 性能测试 7
3. profiling 8
4. 与原算法对比 8
（二） 汇编代码分析 9
1. 矩阵与向量内积 9
2. n 个数求和 9

三、 实验总结和思考 **9**

摘要

本实验围绕 CPU 架构相关编程中的两个基础问题展开：一是 $n \times n$ 矩阵与向量内积，二是 n 个数求和。针对矩阵与向量内积，分别实现了按列访问的平凡算法和按行访问的 cache 优化算法；针对求和问题，分别实现了链式累加、两路链式累加以及两两相加的递归算法三种实现。实验采用固定数据构造方式，以高精度计时器对不同问题规模下的程序平均运行时间进行测量。

实验结果表明：对于矩阵与向量内积问题，若按实验平台信息中给出的 cache 容量选择“未充满 L1、未充满 L2、未充满 L3、基本充满 L3、溢出 L3”五个阶段的规模，按行访问的 cache 优化算法始终优于按列访问的平凡算法；当矩阵规模增至 $n = 2048$（32 MiB，已明显超过 24 MiB 的 L3）时，相较平凡算法获得约 23.00 倍加速。对于 n 个数求和问题，两路链式累加始终优于平凡链式累加，说明减少累加依赖链长度、提升指令级并行能力是有效的。递归算法虽然在理论上具有更短的依赖深度，但在当前单线程实现中由于存在额外复制和多轮中间结果写回，其实际性能未超过两路链式累加。进阶部分进一步表明：循环展开对求和问题收益稳定，而对矩阵问题更多表现为 cache 优化完成之后的次级改进，其效果会随规模和测量环境波动。

本次实验的代码和结果均已整理至 `https://github.com/changluei/Parallel-Programming`。

实验平台信息

项目	说明
操作系统	Microsoft Windows 11 家庭中文版（10.0.26200）
处理器	13th Gen Intel(R) Core(TM) i7-13650HX，14 核 20 线程，最高时钟频率 2.60 GHz
CPU 各级 cache	L1:1.2 MB L2:11.5 MB L3:24.0 MB
内存	13.70 GiB
编译器	`g++`（MinGW），`-O2 -std=c++17`
profiling 编译参数	`-O2 -g -fno-omit-frame-pointer`
绘图环境	Conda 环境 `parallel`，Python 配合 `matplotlib`、`pandas`、`numpy`
profiling 工具	Intel VTune Profiler 2025.10

一、 基础要求

（一） 算法设计

1. $n * n$ 矩阵与向量内积

设输入向量为 $a \in \mathbb{R}^n$，矩阵为 $B \in \mathbb{R}^{n\times n}$，输出向量为 $c \in \mathbb{R}^n$。本实验计算矩阵每一列与向量 a 的内积，即

$$c_j = \sum_{i=0}^{n-1} B_{i,j} a_i, \qquad j = 0, 1, \ldots, n-1.$$

本实验实现了两种算法。

1. **平凡算法（逐列访问）**：外层循环按列遍历，内层循环按行累加，即访问顺序为 `B[row * n + col]`。由于矩阵在内存中按行连续存放，这种按列跨步访问会频繁跨越 cache line，属于 cache 不友好的访问方式。
2. **cache 优化算法（逐行访问）**：先将结果向量清零，然后按行遍历矩阵，将当前行的每个元素与对应的 $a[row]$ 相乘并直接累加到结果向量中。该实现与矩阵的行主序存储方式一致，内层循环访问连续内存，更利于 cache 预取与空间局部性利用。

两种算法的数学结果等价，但由于访存模式不同，在现代处理器上会表现出明显不同的运行时间与 cache 行为。

2. n 个数求和

对于数组 $x[0], x[1], \ldots, x[n-1]$，目标是计算

$$S = \sum_{i=0}^{n-1} x[i].$$

本实验实现了以下三种求和方式。

1. **平凡算法（链式累加）**：维护单个累加器 `sum`，按顺序执行 `sum += x[i]`。该方法实现最简单，但加法之间形成一条很长的数据依赖链。
2. **优化算法（两路链式累加）**：维护两个独立累加器 `sum1` 和 `sum2`，分别累加奇偶位置，循环结束后再合并两个部分和。该方法可以打断单一依赖链，更有利于超标量处理器挖掘指令级并行。
3. **递归思想（两两相加）**：先将相邻两个元素相加得到 $n/2$ 个中间结果，再对中间结果继续两两相加，重复进行 $\log_2 n$ 轮，直到剩下一个结果。本次实验采用二重循环方式实现。

其中，两路链式累加和递归思想都属于对处理器执行特性的利用：前者重点减少串行依赖、增强指令级并行，后者则从规约树结构上缩短依赖深度。

（二） 编程实现

1. $n * n$ 矩阵与向量内积

为了便于实验比较，程序使用固定规则自动生成输入向量和矩阵，为便于测量不同规模的数据，本次采用 `vector` 实现。由于篇幅限制原因，本次不再展示平凡算法的实现。

```cpp
//优化算法实现
fill(sum.begin(), sum.end(), 0.0);
for (int row = 0; row < n; ++row) {//逐行访问矩阵元素
    const double factor = a[row];
    const size_t offset = static_cast<size_t>(row) * n;
    for (int col = 0; col < n; ++col) {
        sum[col] += b[offset + col] * factor;
    }
}
```

Listing 1: 矩阵与向量内积的优化算法核心实现

2. n 个数求和

本次同样采用固定规则生成数据。其核心求和代码如下。

```cpp
//两路链式累加实现
double sum1 = 0.0;double sum2 = 0.0;int i = 0;
const int n = static_cast<int>(data.size());
for (; i + 1 < n; i += 2) {
    sum1 += data[i];
    sum2 += data[i + 1];
}
for (; i < n; ++i) {
    sum1 += data[i];
}
return sum1 + sum2;
//两两相加的递归算法实现，采用二重循环实现
copy(input.begin(), input.end(), work.begin());
for (int m = static_cast<int>(input.size()); m > 1; m /= 2) {
    for (int i = 0; i < m / 2; ++i) {
        work[i] = work[i * 2] + work[i * 2 + 1];
    }
}
return work[0];
```

Listing 2: 两种求和优化核心实现

（三） 性能测试

1. $n * n$ 矩阵与向量内积

本实验采用 `chrono::steady_clock` 作为计时工具，对各算法的核心计算部分进行高精度计时。`steady_clock` 具有单调递增、不受系统时间调整影响的特点，适合用于性能测试。测试时将算法主体封装为独立函数，在主程序中对同一规模下的算法重复执行多次，并以总耗时除以重复次数得到平均运行时间 `avg_ms`，从而减小单次测量误差带来的影响。矩阵实验不再采用简单倍增的规模序列，而是按照实验平台表中给出的 L1=1.2 MiB、L2=11.5 MiB、L3=24.0 MiB 选点，以观察“未充满 L1、未充满 L2、未充满 L3、基本充满 L3、溢出 L3”五个阶段的性能变化。综合“尽量采用 2^n 的幂规模”和“需要覆盖不同 cache 阶段”两点考虑，最终选取 $n = 256, 1024, 1536, 1792, 2048$，对应矩阵大小分别约为 0.50 MiB、8.00 MiB、18.00 MiB、24.50 MiB 和 32.00 MiB。由于不同规模下的运行时间差异较大，重复次数分别设置为 4000、500、250、150 和 100。实验结果如图 1 所示，详细数据见表 1。

规模 n	矩阵大小（MiB）	平凡算法（ms）	cache 优化算法（ms）	加速比
256	0.50	0.032270	0.020948	1.54
1024	8.00	3.844896	0.310954	12.36
1536	18.00	11.484792	1.481136	7.75
1792	24.50	15.754887	1.888080	8.34
2048	32.00	53.424360	2.322430	23.00

表 1: 矩阵与向量内积性能测试结果

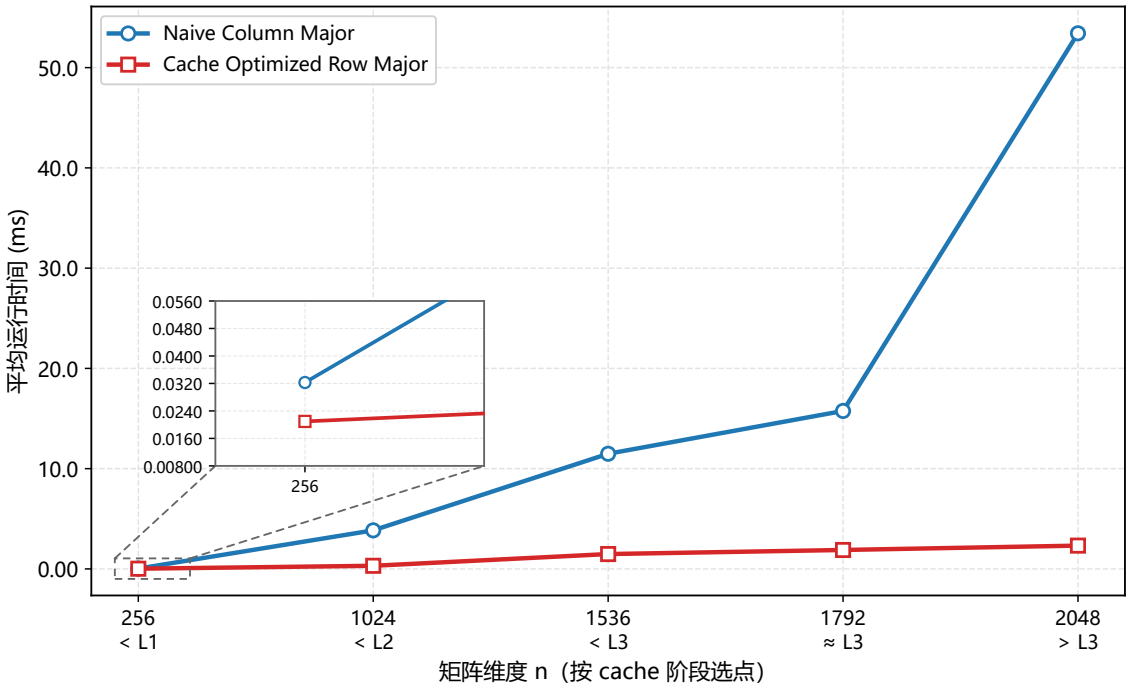

图 1: 矩阵与向量内积运行时间折线图

从图中可以看到，按 cache 阶段选点后，矩阵规模从未充满 L1 逐步增大到溢出 L3 时，平凡算法与 cache 优化算法之间的差距整体上明显拉大，尤其在 24 MiB 附近继续增大到 32 MiB 时出现了更显著的性能跳升。相关具体分析将会在后文展开。

2. n 个数求和

求和实验选择规模 $n = 2^{10}, 2^{11}, 2^{12}, 2^{13}, 2^{14}$，即 1024 到 16384。由于单次求和耗时极短，为了提高计时稳定性，每个数据点均重复执行 1000 次并取平均值。实验结果如图 2 和表 2 所示。

规模 n	平凡链式（ms）	两路链式（ms）	递归算法（ms）	平凡/两路	平凡/递归
1024	0.000411	0.000214	0.000345	1.92	1.19
2048	0.000890	0.000452	0.000587	1.97	1.52
4096	0.001754	0.000855	0.001323	2.05	1.33
8192	0.003638	0.001730	0.003193	2.10	1.14
16384	0.007332	0.003606	0.006953	2.03	1.05

表 2: n 个数求和性能测试结果

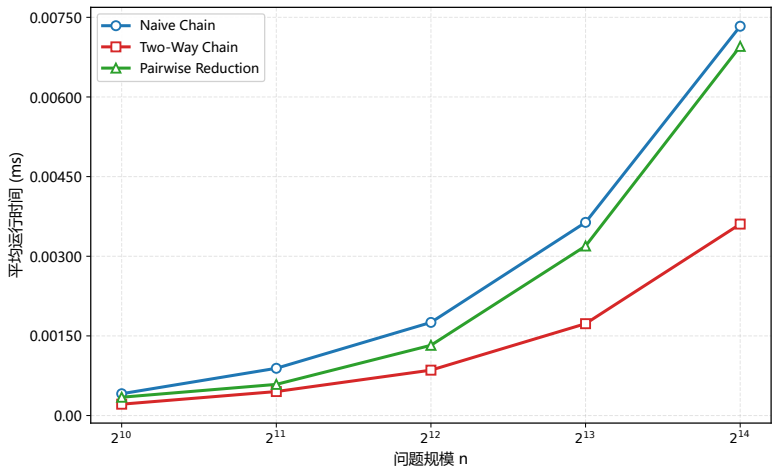

图 2: n 个数求和运行时间折线图

在该实验中，三种算法都能够得到一致的求和结果，但由于其数据依赖关系和中间数据处理方式不同，运行时间存在显著差异。其中，两路链式累加整体最优，说明通过增加独立累加链可以有效提升指令级并行度。至于为何递归算法未能超过两路链式累加，主要原因会在后文进行剖析。

（四） profiling

需要说明的是，小规模测试点运行时间很短，VTune 中个别 miss/hit 计数和比例可能受样本数量限制而波动，因此本文主要利用 profiling 解释总体趋势，而不对少量极端比例作过度解读。

1. $n * n$ 矩阵与向量内积

本次我们对于编译后的程序在 VTune 上测量其三阶段的 cache hit/miss 和 cpu time、CPI。理论上，平凡算法的热点应集中在按列访问的内层循环中，并且会出现更明显的缓存失配；优化算法则由于按行顺序访问矩阵，cache 行为应明显优于平凡算法。

规模 n	cache 阶段	版本	CPI	Cache Miss / Hit			CPU Time (s)
				L1	L2	L3	
256	< L1	平凡算法	1.227	400,006 / 12,000,036	0 / 0	0 / 0	0.014
		优化算法	0.952	0 / 6,000,018	200,042 / 0	0 / 0	0.009
1024	< L2	平凡算法	1.714	2,800,042 / 10,000,030	2,200,462 / 400,006	600,252 / 2,800,588	0.024
		优化算法	0.882	0 / 8,000,024	0 / 400,006	0 / 0	0.013
1536	< L3	平凡算法	1.817	5,200,078 / 18,000,054	5,201,092 / 400,006	1,500,630 / 5,801,218	0.047
		优化算法	0.810	1,200,018 / 22,000,066	400,084 / 1,200,018	200,084 / 0	0.020
1792	≈ L3	平凡算法	1.835	10,800,162 / 26,000,078	9,401,974 / 400,006	1,700,714 / 6,401,344	0.063
		优化算法	0.730	800,012 / 10,000,030	200,042 / 400,006	200,084 / 200,042	0.026
2048	> L3	平凡算法	3.747	11,200,168 / 14,000,042	10,802,268 / 800,012	10,004,200 / 2,200,462	0.166
		优化算法	0.739	1,200,018 / 20,000,060	200,042 / 800,012	400,168 / 0	0.029

表 3: 矩阵与向量内积多规模 profiling 结果记录表

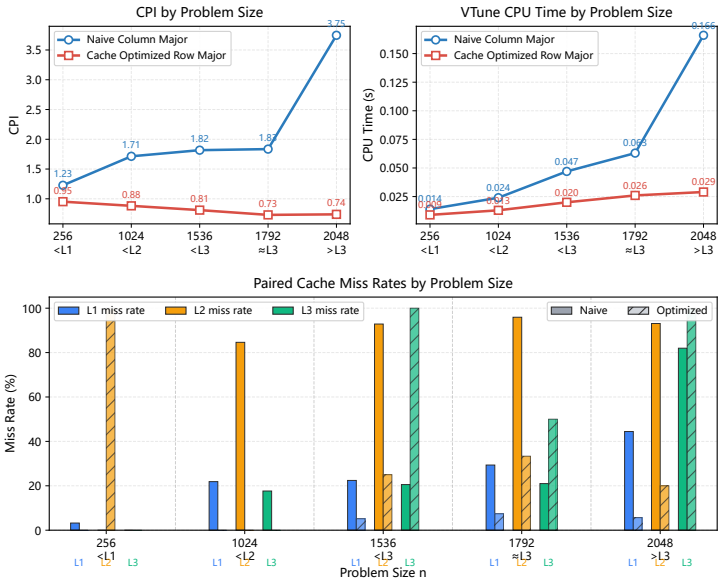

图 3: 矩阵与向量内积多规模 profiling 指标图

2. n 个数求和

对于求和实验，本文采用 VTune 的 `Microarchitecture Exploration` 分析类型，重点关注热点函数、执行指令数（Instructions Retired）、时钟周期数（Clockticks）、CPI 以及 CPU time 等指标。平凡链式累加通常会因为

单条累加依赖链较长而限制指令级并行；两路链式累加有望在保持较低实现成本的同时显著提升吞吐；递归算法则可能在热点中表现出额外的数组复制和多轮规约开销。

版本	热点函数	Instructions Retired	Clockticks	CPI Rate	CPU Time (ms)
平凡链式	`naiveChainSum`	4.20×10^7	3.64×10^7	0.867	7.991
两路链式	`twoWayChainSum`	3.92×10^7	1.68×10^7	0.429	3.995
递归算法	`pairwiseReductionSum`	1.12×10^8	1.96×10^7	0.175	6.992

表 4: n 个数求和 profiling 结果记录表

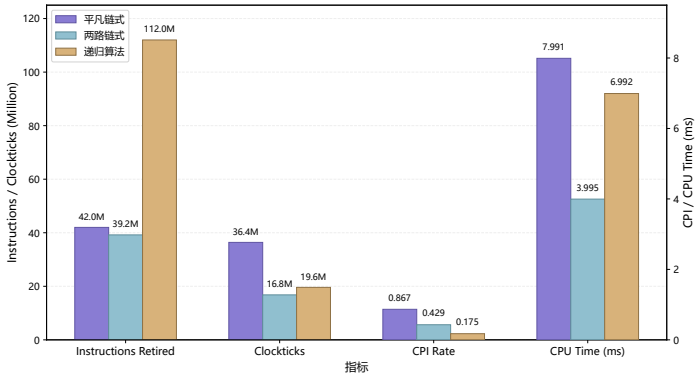

图 4: n 个数求和 profiling 结果图

（五） 结果分析

1. $n * n$ 矩阵与向量内积

矩阵与向量内积的性能变化与 cache 阶段高度相关。$n = 256$ 时工作集很小，平凡算法与优化算法仅差 1.54 倍，说明在数据尚能较好停留在高层 cache 时，访存模式差异虽然存在，但还未被充分放大。到 $n = 1024$ 时，加速比迅速增至 12.36 倍，说明按列跨步访问已经明显破坏空间局部性，而按行访问仍能保持连续访存和较好的预取效果。这里也能看出一个重要事实：性能转折点并不会严格出现在“名义 cache 容量刚好被填满”的位置，因为真实运行中还要受到 cache 可用容量、相联度、替换策略、结果向量读写以及指令和其它数据共同占用空间的影响，因此 $n = 1024$ 虽未完全“装满”L2，却已经表现出明显的层次存储差异。

继续增大到 $n = 1536$ 和 $n = 1792$ 时，加速比回落到 7.75 倍和 8.34 倍，这一“反常”现象并不表示优化失效，而是因为优化算法自身也开始更多依赖更低层 cache：矩阵规模已经接近甚至逼近 L3，按行优化版虽然仍保持顺序访问，但其结果向量更新、写回和更长的数据路径也会带来额外开销；不过它的增长仍远慢于平凡算法。到 $n = 2048$ 时，矩阵明显溢出 L3，平凡算法骤增到 53.424360 ms，而优化算法仅为 2.322430 ms，加速比升至 23.00 倍，说明当工作集全面落入更低层存储后，列访问中“一条 cache line 只利用少量数据”的代价被彻底放大，真正决定性能的仍是访存顺序是否顺应 cache 层次。

profiling 也支持这一判断：平凡算法的 CPI 随规模由 1.227 升至 3.747，而优化算法基本保持在 0.73–0.95，说明前者在规模增大后越来越容易被访存等待拖慢。需要说明的是，优化算法在个别点上出现了偏高的 L2/L3 miss rate，例如 $n = 256$ 时 L2 miss rate 为 100%、$n = 2048$ 时 L3 miss rate 为 100%，这主要是因为这些比例的分母很小，更多反映少量冷启动或流式访问，不能据此认定整体访存质量较差。综合来看，矩阵题的核心不是减少乘加次数，而是让更多访问停留在更高层 cache 中完成；按行优化之所以有效，本质上就是让每次载入的 cache line 被更充分地利用。

2. n 个数求和

三种求和算法的时间都随 n 近似线性增长，但两路链式累加始终最优，相比平凡链式稳定获得约 1.92–2.10 倍加速。原因在于数据始终顺序访问、规模也较小，cache 不是主瓶颈；真正限制吞吐的是单累加器形成的长依赖链。两路链式把累加拆成 `sum1` 和 `sum2` 两条独立路径，在几乎不增加额外工作量的前提下提升了指令级并行度，因此表现最稳定。换句话说，这一问题更接近“算得是否能并行推进”，而不是“数据能否被 cache 命中”。

递归的异常更值得说明。它理论依赖深度只有 $\log_2 n$，如果只看依赖图似乎应当优于链式累加；但本实验采用的是单线程、标量、迭代式规约，需要先复制到 `work`，再多轮读写中间结果，所以总工作量明显增大。profiling 显示它的 CPI 只有 0.175，低于两路链式的 0.429，但 Instructions Retired 却达到 1.12×10^8，约为两路链式的 2.86 倍，因此 CPU time 仍为 6.992 ms，高于后者的 3.995 ms。也就是说，它的问题不是单条指令慢，而是做了太多额外工作；这正说明“依赖深度更浅”并不自动等价于“总时间更短”。

从曲线也能看到这种差异：递归在较小规模时还能略优于平凡链式，因为较浅的规约结构确实能带来一定调度优势；但随着规模增大，多轮遍历、复制和中间结果写回的代价不断累积，这种优势很快被抵消，到 $n = 16384$ 时已只快约 5%。因此，在当前单线程 CPU 场景下，最有效的优化不是单纯追求理论上更浅的规约树，而是像两路链式那样，以很低的实现代价打断依赖链；递归更适合在多线程规约、SIMD 向量化或树形归并场景下发挥优势。

二、 进阶要求

（一） 循环展开优化

1. 算法设计

在基础实验之上，本文继续对已经较为硬件友好的版本实施循环展开（loop unrolling）优化，以进一步减少循环判定、归纳变量更新和分支跳转带来的控制开销。对于矩阵与向量内积，不再对按列访问的平凡版本展开，而是以按行访问的 cache 优化版本为基线，将最内层列循环按 8 个元素展开。这样一来，一次循环迭代中可连续完成 8 组独立的乘加操作，既减少了索引计算和分支判断次数，也为编译器调度和硬件流水线提供了更宽的连续指令窗口；若末尾不足 8 个元素，则仍由标量尾循环完成补齐。对于 n 个数求和，则以两路链式累加为基线，进一步扩展为 4 个累加器，并采用“每轮处理 8 个元素”的方式进行展开。这样既能继续压缩单条依赖链长度，又能在保持实现代价较低的同时增加可并行推进的加法数量。

1. 对矩阵与向量内积的按行访问版本，将内层循环按 8 个元素展开，使一次循环迭代中完成 8 次连续乘加操作，从而减少循环控制开销。
2. 对 n 个数求和的两路链式累加版本继续展开为 4 路累加器，并在单轮循环中处理 8 个元素，进一步增强独立加法链数量。

可参考如下 4 路链式累加展开伪代码。

```
// 以 4 路链式累加为例
sum0 = sum1 = sum2 = sum3 = 0;
for (i = 0; i + 7 < n; i += 8) {
    sum0 += a[i];
    sum1 += a[i + 1];
    sum2 += a[i + 2];
    sum3 += a[i + 3];
    sum0 += a[i + 4];
    sum1 += a[i + 5];
    sum2 += a[i + 6];
    sum3 += a[i + 7];
}
sum = sum0 + sum1 + sum2 + sum3;
```

Listing 3: 4 路链式累加循环展开示意代码

2. 性能测试

在与基础实验相同的数据构造方式下，本文分别对“矩阵按行 cache 优化版”和“n 个数求和两路链式累加版”加入循环展开，并在相同规模与重复次数设置下重新测试平均运行时间。测试结果如表 5 所示。其中，矩阵实验的基线算法为按行访问优化版，展开后版本为 8 路内层循环展开；求和实验的基线算法为两路链式累加，展开后版本为 4 路累加器展开。在正式计时前，展开版本均与对应基线版本进行了结果一致性核对，以确保后续比较反映的是性能差异而非计算结果变化。

问题类型	规模 n	基线算法（ms）	展开后（ms）	加速比	备注
矩阵与向量内积	256	0.006681	0.006646	1.01	位于 L1 阶段，几乎持平
	1024	0.164934	0.188372	0.88	位于 L2 阶段，略慢
	1536	0.818100	0.672508	1.22	超出 L2 后，改善明显
	1792	1.193080	1.125213	1.06	接近 L3，仍有提升
	2048	1.913370	1.505150	1.27	溢出 L3，提升最明显
n 个数求和	1024	0.000328	0.000159	2.06	小规模下收益明显
	2048	0.000450	0.000253	1.78	提升稳定
	4096	0.001130	0.000459	2.46	本组中效果最好
	8192	0.001792	0.001025	1.75	仍有较明显提升
	16384	0.003531	0.002451	1.44	仍优于基线，但收益收窄

表 5: 循环展开优化性能测试结果

从表中可以看出，循环展开对两个问题的收益并不完全相同。对于 n 个数求和，4 路累加器展开在所有测试规模下都优于两路链式累加，说明该问题的主要瓶颈确实与循环控制和依赖链长度有关，展开后可稳定提升吞吐率。对于矩阵与向量内积，8 路展开在 $n = 256$ 时基本持平，在 $n = 1024$ 时甚至略慢，这说明当工作集仍较容易被较高层 cache 容纳时，手工展开带来的代码体积增加和寄存器压力可能抵消部分收益；但在 $n = 1536$、1792 和 2048 等更大规模下，展开版本仍取得了 1.06–1.27 倍的额外加速，表明当访存代价逐渐升高时，减少循环控制开销依然能够带来可观收益。

3. profiling

为分析循环展开是否真正降低了控制开销，本文使用 VTune 对展开前后的代表性规模进行了对比测试。矩阵与向量内积选取 $n = 1024$ 和 $n = 2048$ 两个点，分别对应“仍在较高层 cache 范围内”和“明显溢出 L3”的情形；n 个数求和则选取展开收益较明显的 $n = 4096$ 作为代表规模。对于矩阵题，记录 CPU Time、CPI 和各级 cache miss rate；对于求和题，记录热点函数、Instructions Retired、CPI 和 CPU Time。由于本节重点是对比分析，因此不再单独绘制图像，结果直接整理如表 6 与表 7 所示。

规模 n	版本	CPU Time (s)	CPI	L1 miss rate	L2 miss rate	L3 miss rate
1024	按行优化版	1.635	0.389	31.8%	82.3%	0.2%
1024	8 路展开版	1.958	0.513	27.9%	28.1%	1.1%
2048	按行优化版	1.667	0.995	28.4%	66.3%	82.7%
2048	8 路展开版	1.735	1.111	28.4%	67.7%	87.8%

表 6: 矩阵与向量内积循环展开前后 profiling 对比

规模 n	版本	热点函数	Instructions Retired	CPI	CPU Time (s)
4096	两路链式	`twoWayChainSum`	2.25652×10^{10}	0.755	3.844
4096	4 路展开版	`unrolledFourWaySum`	1.04132×10^{10}	0.460	1.014

表 7: n 个数求和循环展开前后 profiling 对比

4. 与原算法对比

循环展开对两个问题的作用并不相同。对于求和问题，4 路展开相对两路链式改进非常稳定：在 $n = 4096$ 时，CPU Time 由 3.844 s 降至 1.014 s，CPI 由 0.755 降至 0.460，Instructions Retired 也明显减少，说明它同时降低了循环控制开销并进一步缩短了依赖链，因此能稳定转化为实际加速。

矩阵题则更复杂。普通计时中，8 路展开在大规模下有一定收益；但 VTune 下 $n = 1024$ 和 $n = 2048$ 两点的 CPU Time 反而略高，CPI 也上升。尤其在 $n = 1024$ 时，展开版的 L1/L2 miss rate 虽然下降，时间却变慢，这说明异常点并非来自 cache 失配，而更可能与代码体积增大、寄存器压力上升以及对编译器调度的干扰有关。到 $n = 2048$ 时，展开前后 miss rate 已较接近，说明矩阵题的主瓶颈仍是数据供给而非循环分支本身。因此，循环展开对求和是稳定优化，而对矩阵更像 cache 优化之后的次级改进。

（二） 汇编代码分析

下面仅截取热点循环附近的少量汇编示例，用于说明基础实验中的关键差异。

1. 矩阵与向量内积

朴素版（按列访问）

```
imul rdx, rax
call std::vector<...>::operator[] const
call std::vector<...>::operator[] const
mulsd xmm0, QWORD PTR [rbp-72]
addsd xmm0, xmm1
```

按行优化版

```
imul rax, rdx
add rdx, rax
call std::vector<...>::operator[] const
mulsd xmm0, QWORD PTR [rbp-32]
addsd xmm0, QWORD PTR [rbp-152]
```

矩阵题的关键差异不在乘加指令本身，而在地址生成和访存顺序。朴素版在内层循环中反复计算 `row*n+col`，并按列跨步访问矩阵；按行优化版把 `offset=row*n` 提前到外层，只在内层做顺序访问，因此更容易利用 cache line 和预取机制。这与前文“优化主要来自 cache 友好访问”的结论一致。

2. n 个数求和

平凡链式累加

```
movsd xmm0, QWORD PTR [rbp-8]
addsd xmm0, QWORD PTR [rbp-24]
movsd QWORD PTR [rbp-8], xmm0
jne .L21
```

两路链式累加

```
movsd xmm1, QWORD PTR [rbp-8]
addsd xmm0, xmm1
movsd xmm1, QWORD PTR [rbp-16]
addsd xmm0, xmm1
add DWORD PTR [rbp-20], 2
```

求和题的关键差异则是依赖链。平凡链式始终围绕单个累加器更新，同一条加法链严格串行；两路链式把它拆成两条独立路径，处理器更容易并行调度指令，因此在几乎不增加访存量的情况下稳定更快。

综合来看，矩阵题的汇编差异主要反映在“地址生成与访存顺序”上，求和题的汇编差异则主要反映在“是否存在单条长依赖链”上。前者更受 cache 层次结构影响，后者更直接体现了超标量处理器对独立指令流的利用能力，这与前文性能测试和 profiling 得到的结论是相互印证的。

三、 实验总结和思考

本次实验表明，同一数学问题在不同代码组织方式下会表现出完全不同的实际性能，而这些差异往往并不能仅凭算法的渐进复杂度直接看出。对于矩阵与向量内积问题，决定性能的首要因素是访存顺序是否顺应 cache 层次结构：按列访问虽然计算量并未增加，却会显著破坏空间局部性，使大量时间消耗在等待数据上；按行访问则能够更充分地利用 cache line 和预取机制，因此在规模增大、尤其是工作集逐步逼近并超过 L3 cache 后优势迅速放大。对于 n 个数求和问题，数据访问本身始终较为规则，真正限制吞吐率的是单累加器形成的长依赖链；两路链式累加通过引入两条独立的加法路径，提高了指令级并行性，因此能在几乎不增加额外工作量的前提下稳定获得更好表现。进阶实验进一步说明，循环展开并不是对所有问题都同样有效：它在求和问题上收益较稳定，而在矩阵问题上更多属于在访存优化完成之后的次级改进。

从实验方法上看，本次实验也说明了“时间测试、profiling、汇编分析”三者需要结合起来理解程序行为。单纯的运行时间曲线能够回答“是否变快、快了多少”，但不能充分说明性能变化的根本原因；VTune profiling 能从 CPI、Instructions Retired、cache miss rate 等角度解释瓶颈所在；汇编代码分析则进一步揭示了热点循环中地址生成方式、依赖链结构和循环控制开销的差异。与此同时，实验中也存在一些小规模点上的个别异常值、VTune 在短时运行下出现的统计不稳定，以及某些理论上更优的实现因额外工作量过大而未能转化为更好实际性能。这些现象说明，体系结构相关优化并不存在脱离具体平台和实现细节的通用结论。只有把算法结构、内存层次、处理器流水线特性以及实际测量结果结合起来分析，才能更准确地判断一种优化在什么条件下真正有效，这也是本次实验最重要的收获。